AI Empowered Intrusion Detection for MQTT Networks

Steve Chesney
Applied Science & Technology
North Carolina Agricultural & Technical State University
Greensboro, United States
slchesne@aggies.ncat.edu

Kaushik Roy
Computer Science
North Carolina Agricultural & Technical State University
Greensboro, United States
kroy@ncat.edu

2022 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD) | 978-1-6654-8422-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/icABCD54961.2022.9856124

***Abstract*— This paper utilizes the MQTT IDS dataset to demonstrate how artificial intelligence (AI) algorithms can be used for intrusion detection. The MQTT dataset contains both benign and malicious IoT network traffic that includes both TCP and MQTT protocol-level information. The features within the MQTT IDS dataset are labeled for binary classification of cyberattacks, as various attack types were injected in the configured network to simulate real-world conditions. The dataset consists of five recorded scenarios - normal operations and four attack scenarios: (1) Aggressive scan (Scan A), (2) User Datagram Protocol (UDP) scan (Scan sU), (3) Sparta SSH brute-force (Sparta), and the (4) MQTT brute-force attack (MQTT BF). For this study, both traditional machine learning and deep learning techniques were used to classify attacks with a high level of accuracy. For traditional machine learning, the Random Forest, Logistic Regression, Decision Tree, K-Nearest Neighbors, and the Support Vector Machine algorithms were used. For deep learning, a Deep Neural Network (DNN) and a Convolutional Neural Network (CNN) were evaluatedt.**

Keywords—Internet of Things (IoT), machine learning (ML), deep learning (DL), Message Queue Telemetry Transport (MQTT), machine-to-machine (MTM)

I. INTRODUCTION

The recent surge in IoT devices deployed in both consumer and industrial settings has been partly enabled by application layer protocols such as the MQTT (Message Queuing Telemetry Transport) protocol. According to [1], the number of deployed IoT devices increased to 50 billion in 2020. IoT devices take on many forms, from smart devices that are worn by individual consumers, to sensors and actuators that are used in factories and smart cities. In regard to industrial settings (factories), IIoT (Industrial Internet of Things) devices are more optimized for production processes. They connect machines together with communications that don't require human intervention. They support data processing that allow predictive analytics activities [2], such as being able to predict maintenance requirements. These predictions potentially lead to significant cost reductions and to more efficient production systems [2].

One of the reasons for the exponential explosion in IoT devices is because they exist in settings that are more heterogenous than traditional IT networks, which are often called edge or fog computing networks [3]. Moreover, IoT devices extend the reach of traditional computing devices to the edge where physical to virtual abstractions occur. Lastly, IoT devices enable everyday objects to be readable, recognizable, locatable, addressable, and controllable, by means of the Internet [4]. This means that they have the potential of fulfilling the industry goal of being anything, reachable at any time and at any place [5].

As aforementioned, MQTT has become the preferred application layer protocol for the machine-to-machine (MTM) communications that occur within IoT networks. As with many IoT innovations, functionality has been a priority over security, due to the need to respond to market demands. Because of this, many early IoT protocols (such as MQTT) lack sufficient security protections to operate safely in modern environments. The sophistication of cyber-criminals has given rise to the exploitation of vulnerable IoT devices through attacks, such as denial of service (DoS) attacks. DoS and other attack types can overwhelm constrained IoT devices and disable normal communications, as well as gather critical data from device transactions.

The purpose of this paper is to better understand the security vulnerabilities of IoT devices and those which exist in the MQTT protocol. This paper presents an evaluation of different ML & DL techniques that were used to provide intrusion detection against common cyber-attacks against MQTT networks. This analysis was done against the MQTT-IoT-IDS2020 dataset [15]. This paper is organized as follows: Section I – Introduction – A. IoT Protocols, B. IoT Security, C. The MQTT Protocol, and D. MQTT Security; Section 2 – Related Works; Section 3 – Methodology; Section 4 – Experiments and Results; Section 5 – Conclusion.

A. Internet of Things Protocols

IoT devices have well-thought-out communication scenarios, channels and protocols. There are 3 communications scenarios which enable the connectivity of the deployed physical devices: (1) Basic – LAN-based communications, (2) Extended – Internet-based communications, and (3) Cloud – cloud-based communications [6]. Next, there are 4 communications channels: (1) Device-to-device communication (D2D), (2) Device-to-Cloud (D2C), (3) Device-to-Gateway - (D2G), and (4) Backend Data Sharing Pattern [6]. These channels facilitate the different IoT protocols layers which are below [5]:

- Datalink - ZigBee Smart, Bluetooth Low Energy, LoRaWAN, and WiFi, 802.11ah
- Network - 6LoWPAN, 6Lo, IPv6 over Bluetooth Low Energy
- Application - MQTT, SMQTT, AMQP, CoAP, DDS, XMPP

From a functional perspective, IoT devices are deployed in the following layers: (1) perception, (2) network, and (3) application [7, 8]. At the perception layer, IoT devices collect information from their installed environment and/or emit control signals via actuators to other attached

978-1-6654-8422-0/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:28 UTC from IEEE Xplore. Restrictions apply.

mechanisms. The network layer moves the information obtained at the perception layer to the application layer and to the network core infrastructure and vice-versa. Lastly, the application layer, provides customizable services that are dependent upon end-user requirements, acting as a bridge between users and applications. This layer provides highly evolved solutions such as the environmental monitoring [9].

B. *Internet of Things Security*

Although IoT devices have been deployed in large numbers and for many use-cases, they are also opening pathways for exploitation and abuse by cyber criminals [5]. IoT devices remain vulnerable to attacks because of several reasons: they are constrained devices, they exist in large numbers, they are deployed in heterogeneous settings, and because they exist at the edge of networks. IoT devices normally exist on unreliable networks [1], and to exacerbate this reality, vendors often give low priority to security when developing IoT devices to save on cost, time to ship and to enable an easy user experience [1, 6]. Ciklabakkal states in [9] that "IoT devices have little processing capacity, making it difficult to implement a security layer within the device. The higher the encryption algorithm, the more processing is required by the IoT device."

Increasingly, IoT devices have come under various types of cyberattacks. The Mirai Botnet attack is an example. In this attack a distributed denial of service (DDoS) attack was launched from over 400,000 connected IoT devices [3]. There was also the notorious event of a massive DDoS attack, where thousands of security cameras were hacked into remotely, allowing the cyber criminals to take control over a Jeep Cherokee [9].

IoT threats and attacks come from both public (external) and internal IoT networks (if compromised) and are targeted at different IoT protocol layers [1]. This is called the IoT "attack surface" [1]. These different types of attacks are identified by their functional layer below:

- Perception - node capture, malicious code injection, false data injection, replay/freshness, cryptoanalysis and side channel, eaves dropping, interference and sleep deprivation
- Network - DoS, spoofing, sinkhole, wormhole, MiTM, routing information, sybil, unauthorized access
- Application - software attacks - phishing, malicious virus/worms and malicious scripts

Since IoT devices are constrained, typical cyber software and network protection packages and algorithms cannot be used effectively against attacks; instead, ML and DL solutions are needed to bolster IDS systems [10, 11]. IoT devices require an intrusion detection system, much like those required for traditional networks. Yet, IoT devices and networks have specialized architectures that require highly adaptable IDS solutions. Intrusion detection systems are normally divided into two classes: signature-based and anomaly-based [11]. According to Leal, intrusion detection systems can be implemented in 3 ways – centralized, hybrid or distributed [7]. Also, IDS's can be packet-level based or flow-based. Lastly, IDS methods (also called classes) are anomaly, signature, hybrid, or specification based.

C. *Message Queue Telemetry Transport*

With the number of IoT devices having grown so rapidly, machine-to-machine (MTM) communications have become a necessity, as direct management by human-enabled systems is not feasible. This has caused the implementation of the MQTT protocol to be standardized [1], especially where fleet-deployments (installation en masse) is the normal use-case. [2] confirms that MQTT is now widely used in both consumer IoT and in IIoT (Industrial IoT) contexts.

The MQTT protocol has several key features and capabilities that have caused its escalated adoption by developers and users, namely, a low bandwidth requirement, low memory consumption and a lightweight payload; it is a resource-constrained protocol [3, 5]. According to [12], MQTT protocol adoption has outgrown other similar IoT application-layer protocols. AMQP (Advanced Message Queuing Protocol), CoAP (Constrained Application Protocol), DDS (Data Distribution Service) and XMPP (Extensible Messaging and Presence Protocol) are examples of other IoT application layer protocols.

MQTT is a session layer protocol (OSI model) but in the TCP network model, MQTT is in the Application layer [4]. According to [13] MQTT was invented in 1999 by Arlen Nipper (wrote an article) and Dr. Andy Stanford-Clark (IBM) while working on a project for oil pipelines. They needed to sense and act on oil pipelines, but the connectivity was inefficient and expensive to satellites. There were also unique characteristics to the communications, such as simple implementation, quality of service data delivery, lightweight and bandwidth efficient, data agnostic, continuous session awareness [13]. These initial requirements are the same for current IoT applications. The OASIS (Organization for the Advancement of Structured Information Standards), ISO/IEC 20922 standard states: "MQTT is a Client Server publish/subscribe messaging transport protocol. It is lightweight, open, simple, and designed so as to be easy to implement. These characteristics make it ideal for use in many situations, including constrained environments such as for communication in Machine to Machine (M2M) and Internet of Things (IoT) contexts where a small code footprint is required and/or network bandwidth is at a premium" [5].

The main characteristics of the MQTT protocol are its binary format, its efficiency (even in unreliable scenarios), its bi-directional communications capabilities, its ability to scale, its publisher-broker-subscriber architecture, and its ability to run on constrained devices [13]. The publish-broker-subscribe pattern has 3 main components: (1) an MQTT Client (publisher), (2) an MQTT Broker and (3) an MQTT Subscriber (also a client). Within this architecture, the publisher, publishes a message to the broker. The broker, which at the heart and center of each communication, is also the orchestrator of the communications cycle. The MQTT Client (Subscriber) receives messages from broker (the client can also be publishers because of bi-directional communication), after it subscribes to a Topic. Within "pub/sub" pattern, communications are decoupled. Clients are decoupled spatially (they are not bound by geography or time) [13]. Lastly, an MQTT session is divided into 4 phases:

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:28 UTC from IEEE Xplore. Restrictions apply.

connection, authentication, communication, and termination [2].

D. MQTT Security

Despite the MQTT protocol's salient features, it remains a technology where insufficient security protections were embedded in its functionality. [1] states that MQTT only provides authentication, but no encryption of communications and data. MQTT primarily focuses upon message transmission and the burden is upon the implementer to provide security features (normally done through TLS) [5].

The main vulnerability of the MQTT protocol is its openness (lack of robust authentication, and absence of encryption) which can be exploited, by DoS attacks [11]. The main objective of a DoS attack against an MQTT environment is to flood the broker server, thus disallowing communications between the publishers and subscribers of the service. By controlling the broker server, attackers can also publish false messages, alter existing message and sniff/analyze messages that are exchanged between publishers and subscribers through exchanged topics [11].

There are several main attack scenarios for MQTT-enabled devices [1]: Blackbox (information gathering) attacks, Shodan API Search visibility of public MQTT brokers, which have a connection code of "0" (Shodan is an open API that reveals publicly accessible devices), data privacy leakages (MQTT provides not native packet encryption), authentication attacks (CONNECT messages show plaintext username and passwords), and KeepAlive messages can be manipulated to invoke the resending of CONNECT messages, in which the username and password can be collected.

[4] states that the MQTT protocol has other security challenges, which are listed below:

- Any client can subscribe to any topic with minimal validation and verification
- Brokers can become "bogged down" if a client does not collect messages from the broker
- Messages are not prioritized, so critical messages can be delivered without urgency over "normal" messages
- Ordering and resending of messages can be list during transmission
- MQTT often struggles with handling of large messages
- The usage of "wildcards" in topic names may allow attackers to steal data
- The most subscribed topic can be viewed if the broker is compromised, allowing a DoS attack to flood the broker

II. Related Works

As aforementioned, this work focuses upon the analysis of the MQTT-IoT-IDS2020 [15] through ML and DL methods. There has been other recent research done against similar datasets that have yielded very positive results for the detection and classification of MQTT-focused cyberattacks. Additionally, signature-based approaches, public search API countermeasures (Shodan) and certificate-based protection approaches have been used to safeguard IoT environments.

In [1] a deep learning-based Network Intrusion Detection System (NIDS) was trained using the MQTT-IoT-IDS2020 dataset. Accuracy, F1, Recall, Precision and other metrics were used to evaluate the effectiveness of DNN, CNN-RNN-LSTM, LSTM and Decision Tree algorithms. Each of these models exhibited similar behavior and displayed a high-level of accuracy in detecting these attacks.

In [15] an MQTT-focused dataset was created with the following attack types: Aggressive Scan, User Datagram Protocol Scan, Sparta SSH Brute-force attack, and a MQTT Brute-force attack. The effects of these attacks on both TCP and MQTT packet parameters were observed and evaluated for intrusion detection. The machine learning algorithms used to detect these attacks with different levels of accuracy were Logistic Regression, KNN, Decision Tree, Random Forest, Support Vector Machine (RBF Kernel), and Support Vector Machine (Linear Kernel).

In [2] the MQTTset dataset was used to evaluate the effectiveness of SlowITe (a slow rate DoS attack) against the broker server of an MQTT environment. SlowITe attacks are often difficult to attack because they do not elevate the usage of CPU and memory resources in IoT devices. According to [2], means of protection against SlowITe attacks are still an open area for research, but several solutions have been suggested, which include the ARTEMIS IDS seen in [9].

In [10], a DNN was proposed for intrusion detection and compared against several traditional machine learning algorithms: Naïve Bayes, Random Forest, KNN, Decision Tree, LSTM, and Gated Recurrent Units. In this work, the dataset in [3] was also investigated with similar results. Varying results were achieved with the different ML and DL algorithms and data subsets.

In [14], a dataset to developed which is focused on the MQTT protocol. The data contained within it captures both normal and malicious traffic, showing the effects of cyberattacks on TCP and MQTT packet data. The MQTT broker was the target of the attacks, while Neural Networks, Random Forest, Naïve Bayes, Decision Tree, Gradient Boost and a Multilayer Perceptron were evaluated for attack detection. The following attack types were launched: Flooding DoS, MQTT Publish Flood, SlowITe, Malformed Data and Brute Force Authentication.

In [3], a dataset was developed which includes DoS, MiTM and an Intrusion attack (using port 1883 – MQTT - to understand the active topics of the MQTT network) attack data against an MQTT environment. For attack detection and classification, the following algorithms were used: XGBoost, RNN, an LTSM Recurrent Network, and a GRU Recurrent Network were used. The XGBoost method provided the highest accuracy, where it was concluded the ensemble methods showed the best for intrusion detection.

In [9] a dataset was developed which was made publicly available. This dataset contains information from the TCP, IP and MQTTT layers to show the effects of various cyberattack types. The following ML algorithms were used and evaluated to create an IDS called "Artemis": AutoEncoders, Single-Objective Generative Adversarial

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:28 UTC from IEEE Xplore. Restrictions apply.

Active Learning (SO_GAAL), Random Forest, Isolation Forest, One-Class Support Vector Machine (OCSMV) and K-Means Clustering.

In [5] vulnerabilities of MQTT communications are analyzed when exposed cyberattacks. Particularly, the effects on the message payload are observed when a MiTM attack is launched. 2 different types of security measures are experimented with to secure the payload – transport layer and application layer. The application layer security scheme using AES (Advanced Encryption Service) proved to be the successful implementation against MiTM attacks.

In [11] a fuzzy logic-based countermeasure was used to protect MQTT communications against a DoS attack. The fuzzy-logic-based protection scheme was given the name "Secure-MQTT", as it detects malicious behavior using fuzzy rule interpolation, which generates detection rules dynamically. Secure-MQTT uses correlation-based network feature selection and then a fuzzy-logic engine developed in the broker server determines the presence of malicious activity.

In [4], a Python-based search engine and API is used to determine if MQTT brokers are publicly accessible. Then, a Python program was used to connect to a located server that allowed unauthorized access. Next, a command was executed to show a topic which revealed all system data. Once the vulnerable server was identified and the exploited security mechanisms, configuration files were modified, and ACLs (Access Control Lists) were created to prevent unauthorized access. Once these security mechanisms were implemented, cyberattacks against the MQTT broker were unsuccessful.

III. Methodology

A. Machine Learning & Deep Learning Algorithms

For this work, the software tools contained within the Python 3.8 and the Scikit-Learn machine learning library, version 0.23.2 was used. The target dataset was the MQTT-IoT-IDS2020 dataset [15], which was first cleaned to eliminate missing values, and then object (string) values we converted to numerical values. Categorial values were also transformed to binary values via the use of Scikit's Learns OneHot Encoding feature. During the preprocessing phase, StandardScaler was utilized to scale the numeric data values between 0 and 1. Next, the Shuffle class was used to mix the different attack types within the dataset. Afterwards, the data was split into a set of features (X), and into a series of target values (y values) for processing within the K-Fold Cross Validation (with 5 folds) class. An object was made of this class to partition the dataset into 5 folds, with each fold, being used a new testing subset for each new iteration.

Lastly, the Random Forest ML algorithm was used for feature selection to identify the attributes of the dataset which were most important in identifying the presence of MQTT attacks. The features below were identified as the most important: 'ip_length', 'tcp_flag_push', 'mqtt_messagetype', 'mqtt_messagelength' and the 'mqtt_flag_clean.'

In this work, different machine learning and deep learning techniques were used. Both the ML and DL model were evaluated for binary classification effectiveness against the MQTT-IoT_IDS2020 dataset. The traditional machine learning algorithms list below were evaluated:

- Random Forest
- Logistic Regression
- Decision Tree
- K-Nearest Neighbors
- Support Vector Machine (SVM)

For deep learning, the Keras library, version 2.5.0 with a TensorFlow backend was used to create a Deep Neural Network (DNN). This DNN was comprised of four layers - 3 dense layers (including the input layer), and one output layer. The dense layers contained 32, 64 and 128 neurons, respectively. Each of the dense layers utilized the ReLU activation function, while the output layer utilized the sigmoid function for binary classification. When the DNN model was compiled, the Adam optimizer was used, along with Binary Crossentropy as the loss function. Lastly, when the fit function was called, 10 epochs were run to achieve the highest level of accuracy, with a batch size of 128.

A CNN was also evaluated against the MQTT-IoT-IDS2020 dataset via a Sequential model. Alike the DNN, the Keras library, version 2.5.0 with a TensorFlow backend was used. A one-dimensional input layer was added with 64 filters (neurons), a kernel size (the size of the filters) of 2; the ReLU activation function was used, along with the input shape that matched the dataset's dimensions. Next, another one-dimensional layer was added with the following arguments: 32 filters, kernel size of 3 and the ReLU activation function. Finally, a flattening layer was added to flatten the input shape with an output layer that used the Sigmoid function for binary classification.

B. Dataset Details

This work utilized the MQTT-IoT-IDS2020 dataset [15] which contained the following features (significant features after feature selection), output label, cyber-attack types and dataset shapes:

TABLE 1. MQTT-IoT-IDS2020 Key Features and Output (Label)

Dataset Features & Labels	**Significant Features / Label**					
	ip_len	***tcp flag push***	***mqtt message type***	***mqtt message length***	***mqtt flag clean***	***is attack (label)***
Data type	float64	float64	float64	float64	float64	binary

[a] The MQTT-IoT-IDS2020 dataset contains TCP and MQTT traffic packet captures.

TABLE 2. MQTT-IoT-IDS2020 Key Feature & Label Descriptions

Dataset Features & Labels	**Feature / Label Description**
ip_len	The length of ip packet including header & data
tcp_flag_push	Tells client/server to send data to application even if buffer is filled up.
mqtt_messagetype	Message type used for various communications activities

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:28 UTC from IEEE Xplore. Restrictions apply.

mqtt_messagelength	The size of queue for messages in persistent sessions
mqtt_flag_clean	The state of persistent sessions
is_attack	0 for benign or 1 for cyber attack

TABLE 3. MQTT-IoT-IDS2020 Cyber Attack Types

Dataset	Cyber Attack Types			
	Brute Force	***Sparta SSH Brute Force***	***Aggressive Scan***	***UDP Scan***
Rows	10,045,316	20,676,140	111,392	233,255
Columns	31			

[b.] There were 31 columns initially, but they were reduced to 5 columns after feature selection.

c.

IV. Experiments and Results

The table below shows the results of the different ML and DL algorithms evaluations in their ability to detect the different MQTT cyber-attack types. Firstly, all algorithms were very effective in the binary classification experiments. Secondly, most of the algorithms showed the least accuracy in classifying the Sparta SSH attack type; however, both DNN and CNN showed their least accuracy in detecting the Aggressive Scan attack. Lastly, the traditional ML algorithms showed very similar results to those obtained from the DL algorithms.

TABLE 4. Machine Learning & Deep Learning Evaluation Results

Attack Types	**Algorithms & Evaluation Metrics**				
	Algorithm	***Accuracy***	***Precision***	***Recall***	***F1 Score***
Brute Force	Random Forest	99.97	99.98	99.99	99.98
	Logistic Regression	99.35	99.94	99.98	99.96
	Decision Tree	99.92	99.92	1.00	99.96
	K-Nearest Neighbors	99.97	99.96	1.00	99.98
	SVM	99.97	99.99	99.97	99.98
	DNN	99.97	99.77	1.00	99.89
	CNN	99.79	99.79	1.00	99.89
Sparta SSH	Random Forest	97.17	97.12	1.00	98.53
	Logistic Regression	96.77	96.73	1.00	98.35
	Decision Tree	97.12	97.05	1.00	98.51
	K-Nearest Neighbors	97.17	97.12	1.00	98.53
	SVM	96.77	96.73	1.00	98.34
	DNN	95.05	95.05	1.00	97.46
	CNN	94.97	94.97	1.00	97.42
Aggresive Scan	Random Forest	99.95	99.98	1.00	99.99
	Logistic Regression	99.87	99.98	1.00	99.84
	Decision Tree	99.95	99.98	1.00	99.81
	K-Nearest Neighbors	99.95	99.98	1.00	99.99
	SVM	99.91	99.98	99.75	99.87
	DNN	63.16	0.00	0.00	0.00
	CNN	63.53	0.00	0.00	0.00
UDP Scan	Random Forest	99.95	1.00	1.00	99.99
	Logistic Regression	99.89	99.98	1.00	99.85
	Decision Tree	99.98	1.00	1.00	99.93
	K-Nearest Neighbors	99.95	1.00	1.00	99.99
	SVM	99.91	99.98	99.97	99.95
	DNN	90.42	0.00	0.00	0.00
	CNN	90.36	0.00	0.00	0.00

V. Conclusion

The MQTT protocol was originally created to facilitate remote communications over unreliable connections; and although it has been rapidly adopted (for that reason) for use in modern IoT networks, it lacks robust security mechanisms. This lack of security (internal to the protocol itself) has made MQTT communications vulnerable to cyber-attacks. This paper has shown that both machine learning and deep learning techniques can be used to provide security for constrained, MQTT-based IoT networks.

For future work, more investigation should be conducted to reveal why the Sparta SSH attack type is the least accurately classified attack amongst most algorithms and why both DNN's and CNN's score very low when detecting the Aggressive Scan attack type. Next, Additional DL algorithms should be tested against the MQTT-IoT-IDS2020 dataset, such as AutoEncoders, RNN's, and LTSM's. Another area of focus should be the labeling of the MQTT-IoT-IDS2020 dataset for multiclass classification. This will go beyond testing the effectiveness of ML and DL algorithms for identifying the presence of an attack, but also determine their effectiveness in classifying attack types. Lastly, both the machine learning and deep learning algorithms used in this paper should be conducted against other publicly available MQTT datasets, such as the MQTTset [14] and the 'joseaveleira_es_dataset' [3].

Acknowledgments

This research was funded by a National Centers of Academic Excellence in Cybersecurity (NCAE-C) Grant (H98230-21-1-0326), which is part of the National Security Agency.

References

[1] S. Andy, B. Rahardjo and B. Hanindhito, "Attack scenarios and security analysis of MQTT communication protocol in IoT system," In the 4th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI), September 21, 2017. IEEE. Available: DOI: 10.1109/EECSI.2017.8239179.

[2] I. Vaccari, "SlowITe, a Novel Denial of Service Attack Affecting MQTT," May 21, 2020, *Sensors*., vol. 20, (10).

[3] Hector Alaiz-Moreton, "Multiclass Classification Procedure for Detecting Attacks on MQTT-IoT Protocol," *Complexity*, vol. 2019, 2019.

[4] M. Harsha, B. Bhavani and K. Kundhavai, "Analysis of vulnerabilities in MQTT security using shodan API and implementation of its countermeasures via authentication and ACLs," In 2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI), 2018, DOI: 10.1109/ICACCI.2018.8554472.

[5] Varma Arunima, "Effect of payload security in MQTT protocol over transport and application layer," *IOP Conference Series: Materials Science and Engineering*, vol. 1166, (1), 2021.

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:28 UTC from IEEE Xplore. Restrictions apply.

[6] S. N. Firdous et al, "Modelling and evaluation of malicious attacks against the IoT MQTT protocol," In 2017 IEEE International Conference on Internet of Things (iThings), 2017, DOI: 10.1109/iThings-GreenCom-CPSCom-SmartData.2017.115.

[7] R. Leal; L. Santos; L. Vieira; R. Gonçalves and C. Rabadão, "MQTT flow signatures for the internet of things," In 2019 14th Iberian Conference on Information Systems and Technologies (CISTI), 2019 .DOI: 10.23919/CISTI.2019.8760849.

[8] M. Bilal, "A Review of Internet of Things Architecture, Technologies and Analysis Smartphone-based Attacks Against 3D printers," *arXiv*, 2017.

[9] E. Ciklabakkal, "ARTEMIS: An intrusion detection system for mqtt attacks in internet of things," in Proceedings of the IEEE Symposium on Reliable Distributed Systems, pp. 369-371, 2019.

[10] M. Khan, "A Deep Learning-Based Intrusion Detection System for MQTT Enabled IoT." *Sensors*. Basel, Switzerland, vol. 21, 2021.

[11] A. Haripriya and K. Kulothungan, "Secure-MQTT: an efficient fuzzy logic-based approach to detect DoS attack in MQTT protocol for internet of things," *EURASIP Journal on Wireless Communications and Networking*, vol. 90, pp. 1-15, 2019. Available: https://jwcn-eurasipjournals.springeropen.com/articles/10.1186/s13638-019-1402-8.

[12] I. Skerrett. Why MQTT has become the de-facto IoT standard. Available: https://iskerrett.medium.com/why-mqtt-has-become-the-de-facto-iot-standard-9bed15588cc9.

[13] D. Obermaier, *MQTT & MQTT 5 Essentials: A Comprehensive Overview of MQTT Facts and Features for Beginners and Experts Alike*. Landshut, Germany: HiveMQ, 2020.

[14] I. Vaccari, "MQTTset, a New Dataset for Machine Learning Techniques on MQTT." *Sensors.* Basel, Switzerland, vol. 20, (22), 2020.

[15] H. Hindy, E. Bayne, M. Bures, R. Atkinson, C. Tachtatzis and X. Bellekens, "Machine Learning Based IoT Intrusion Detection System: An MQTT Case Study (MQTT-IoT-IDS2020 Dataset)," In Conference Proceedings of the 12th International Networking Conference, January 5, 2021, United Kingdom. Springer, 2021. Available: https://link.springer.com/chapter/10.100/978-3-030-64758-2_6

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:47:28 UTC from IEEE Xplore. Restrictions apply.